The OSI Model for System Design

L3, L4, L7, and where TLS really sits

Watch the 3-min video

In one line: the OSI model splits networking into seven layers, each doing one job; in system design you mostly care about L3 (IP), L4 (ports, TCP/UDP) and L7 (HTTP), because that is what "L4 load balancer" and "L7 firewall" refer to.

1 Seven layers, one job each

You type a URL and a page appears in milliseconds. Behind that, a stack of protocols is working. Each layer does one job and relies on the layer below it. The **OSI model** gives each layer a number, and engineers use those numbers as shorthand all the time.

From the bottom: **1 Physical** sends bits on wires and radio. **2 Data link** moves frames between neighbours on one network, using MAC addresses. **3 Network** routes packets between networks using IP addresses. **4 Transport** connects processes using ports, with TCP or UDP. **5 Session**, **6 Presentation** and **7 Application** sit on top; HTTP lives at layer 7. To remember the order from the top: *All People Seem To Need Data Processing*.

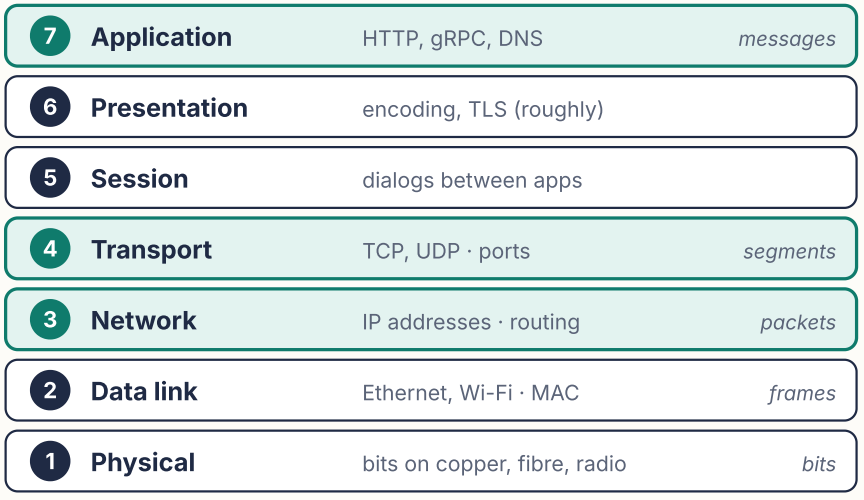
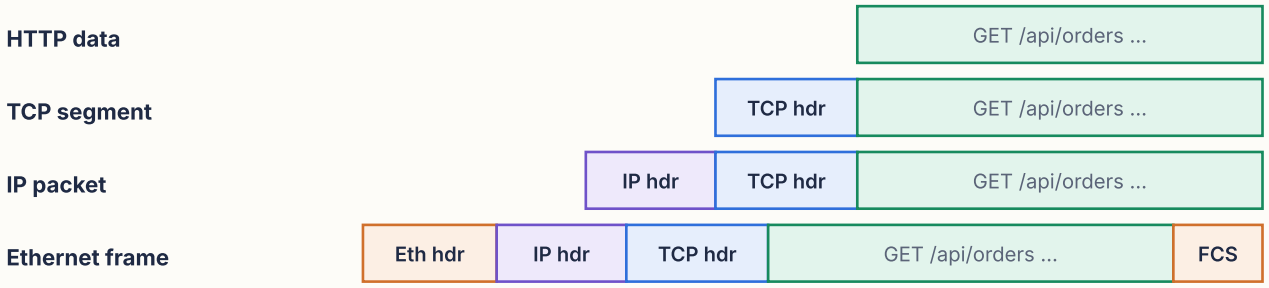


Figure 1. The OSI stack. Layers 3, 4 and 7 (highlighted) are the ones you will name in system design interviews.

Why only three? Layer 3 decides *which machine* gets a packet. Layer 4 decides *which process* on that machine, and whether delivery is reliable (TCP) or fast and best-effort (UDP). Layer 7 is the first layer that understands the actual request: the method, the path, the headers and the cookies.

2 Encapsulation: wrapping on the way down

As data goes down the stack, each layer wraps it in its own header. HTTP data becomes the payload of a **TCP segment**, which becomes the payload of an **IP packet**, which goes inside an **Ethernet frame**, which is sent as bits. The receiver unwraps it in reverse order. Each layer only reads its own header and treats the rest as opaque bytes. This is **encapsulation**.



sender: each layer adds its own header on the way down · receiver: strips them in reverse on the way up

Figure 2. The same HTTP request at each layer. Each header carries what that layer needs: ports (TCP), IP addresses (IP), MAC addresses and a checksum (Ethernet).

3 Every device works at a layer

A device's layer tells you what it can see, and so what it can decide on:

Device	Layer	What it looks at
Switch	L2	MAC addresses; forwards frames inside one network
Router	L3	Destination IP; forwards packets between networks
Basic firewall, cloud security group	L3 / L4	IPs, ports and protocol (allow TCP 443 from 10.0.0.0/16)
L4 load balancer	L4	IPs and ports of a connection
Reverse proxy, API gateway, WAF, L7 LB	L7	The HTTP request itself: method, path, host, headers, cookies, body

4 L4 vs L7 load balancers

This is the classic interview example. An **L4 load balancer**, such as the AWS Network Load Balancer, sees only IP addresses and ports. It picks a backend for each connection and forwards the bytes without reading them. That makes it very fast, and it works for any TCP or UDP protocol: databases, MQTT, game traffic, gRPC, anything.

An **L7 load balancer**, such as the AWS Application Load Balancer, reads HTTP. It can send `/api` to one service and `/images` to another, route by host name or header, add headers, and retry a failed request on another backend. The price is more work per request: it must parse every request, and usually decrypt it first.

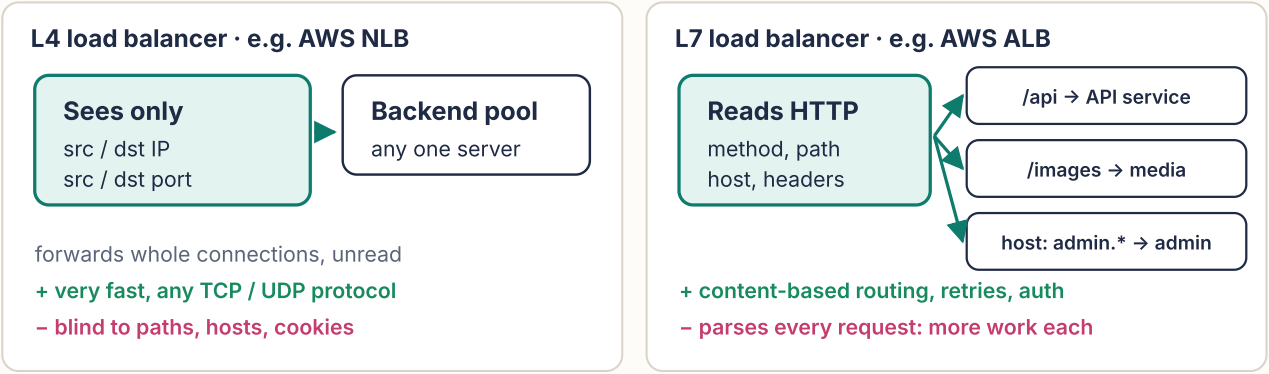


Figure 3. Left: an L4 balancer forwards connections by IP and port only. Right: an L7 balancer reads each HTTP request and routes it by path, host or header.

WORKED EXAMPLE: WHICH ONE?

A public website with /api, /images and /admin served by different services: L7. Only something that reads the path can split them, and you get one TLS certificate in one place.

A fleet of PostgreSQL read replicas, or a multiplayer game on UDP: L4. There is no HTTP to read, and you want the lowest added latency. Many real setups use both: an L4 balancer in front for raw scale and a static IP, L7 proxies behind it for routing.

5 Where does a TLS-terminating proxy operate?

TLS sits above TCP and below HTTP. In OSI terms that is roughly the presentation layer (6), and the TCP/IP model just counts it as part of the application. But once a proxy has decrypted the traffic, it can read the HTTP inside, so in practice we call a TLS-terminating proxy **layer 7**. The alternative is **TLS passthrough**: an L4 balancer forwards the encrypted bytes untouched and the backend decrypts them. The balancer then never sees the path or the headers.

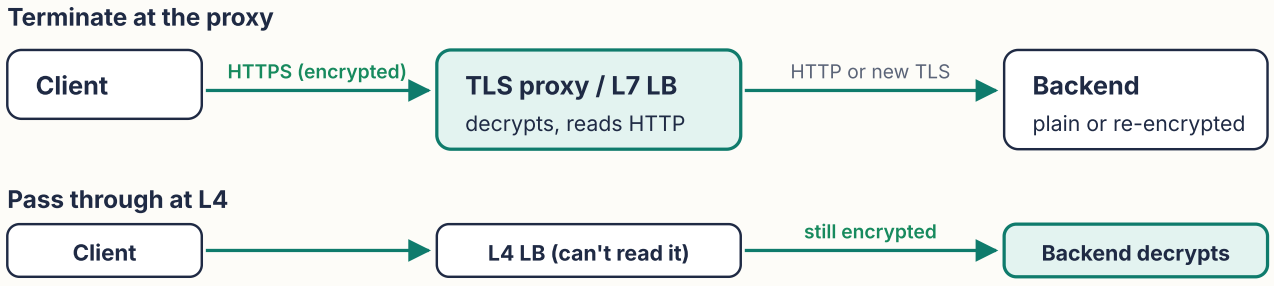


Figure 4. Terminating TLS at the proxy lets it route by HTTP content; passing it through keeps traffic encrypted end to end, but the balancer can only route by IP and port.

COMMON MISTAKES

- Saying an L4 balancer can route by URL path: it never sees the path.
- Calling TLS “layer 4” because it runs over TCP. It sits above TCP; a proxy that decrypts it acts at L7.
- Treating OSI as how the internet is built. It's a vocabulary; the internet runs on the 4-layer TCP/IP model (next lesson).
- Forgetting that L7 features need decryption, so the proxy needs the certificate and private key.

6 Key terms

OSI model: 7-layer reference model for networking

Port: number that picks a process on a host (L4)

TLS termination: proxy decrypts, then can read HTTP

Encapsulation: each layer wraps data in its own header

L4 load balancer: routes connections by IP and port

TLS passthrough: balancer forwards encrypted bytes; backend decrypts

MAC address: hardware address used at L2

L7 load balancer: routes requests by HTTP content

WAF: web application firewall, filters HTTP at L7

7 Interview questions

Q1. Name the OSI layers and the one or two that matter most in system design. BASIC

Physical, data link, network, transport, session, presentation, application. The ones that come up most are L3 (IP routing), L4 (TCP/UDP and ports) and L7 (HTTP: method, path, headers).

Q2. At which layer does a TLS-terminating proxy operate? BASIC

TLS itself sits between transport and application (roughly OSI layer 6). Once the proxy decrypts, it reads HTTP, so it operates at layer 7 in practice. An L4 balancer can instead pass the encrypted bytes through for the backend to decrypt.

Q3. Why does the L4 vs L7 distinction matter for load balancers? BASIC

L4 sees only IPs and ports: very fast, works for any TCP or UDP protocol, but blind to content. L7 reads HTTP, so it can route by path, host and headers, retry requests and terminate TLS, at a higher cost per request.

Q4. You need /api/* to go to the API service and /static/* to a file server. Which load balancer? SCENARIO

An L7 load balancer (ALB, NGINX, Envoy). Routing by path needs to read the HTTP request, which an L4 balancer never sees.

Q5. You must load-balance 100,000 long-lived MQTT or game connections with minimal added latency. Which type? SCENARIO

L4. The traffic isn't HTTP, connections are long-lived, and an L4 balancer forwards packets with almost no per-request work. A managed NLB scales to millions of connections and can keep the client's source IP.

Q6. Security wants traffic encrypted all the way to the app servers, but you also want path-based routing. What do you do? SCENARIO

Terminate TLS at the L7 proxy to read the path, then re-encrypt to the backends with a new TLS connection. Passthrough would keep one TLS session end to end but loses path routing.

Q7. A security group allows TCP 443. Can it block a request to /admin? TRICKY

No. Security groups filter at L3/L4 (IP, port, protocol) and never see the URL. Blocking /admin needs an L7 component: a WAF, API gateway or reverse proxy rule.

Q8. Is an L7 load balancer always better because it has more features? TRICKY

No. It parses (and usually decrypts) every request, adding latency and CPU, and only handles protocols it understands. For non-HTTP traffic or raw throughput, L4 is the better tool; many systems put L4 in front of L7.

Next: SD-B-002 · The TCP/IP Model backendroadmap.com